

AviPack

Versioned Project Brain Starter for AI-Assisted Software Development

1. Overview

AviPack is a universal, installable project starter that adds a versioned **project brain** to any software project.

It is designed for teams, solo developers, founders, and freelancers who use AI coding agents to develop software. AviPack helps AI agents continue development according to approved requirements, architecture, sprint scope, QA rules, and release gates.

AviPack is not a coding agent itself.
It is a **control and governance layer** for coding agents.

It helps tools such as Codex, Claude Code, Cursor, GitHub Copilot, Devin, Windsurf, Replit Agent, and other AI development assistants work inside a clean, traceable, and controlled software development lifecycle.

2. One-Line Description

AviPack is a versioned project brain starter that keeps AI coding agents aligned with requirements, architecture, sprint scope, QA, and release rules.

3. Tagline Options

- **A project brain for AI-assisted development.**
- **Keep AI agents aligned with your project scope.**
- **Versioned requirements, scoped development, controlled AI execution.**
- **The project brain pack for modern AI development.**
- **From idea to scoped AI execution.**

Recommended tagline:

A project brain for controlled AI-assisted software development.

4. Problem

AI coding agents can write code effectively, but they often need constant human guidance.

Common problems include:

- AI follows old or incomplete requirements.
- AI mixes chat instructions with approved scope.
- AI touches unrelated files.
- AI adds unnecessary features or refactors.
- AI misses QA coverage.
- AI does not understand sprint boundaries.
- AI changes architecture without approval.
- AI builds the right code for the wrong requirement version.
- Human owners must repeatedly review and correct small deviations.

The real issue is not only coding ability.

The real issue is:

AI agents need a reliable, versioned, and machine-readable project brain.

5. Solution

AviPack adds a structured project brain to any repository.

This project brain becomes the source of truth for:

- Product vision
- Project context
- Requirements
- Requirement versions
- Change requests
- Designs
- Architecture decisions
- Sprint scope
- Sprint locks
- QA mapping
- Test coverage
- Release readiness
- Human approval rules
- AI agent behavior rules

Before an AI agent starts development, it must read the project brain and understand:

- What requirement is active
 - What version is approved
 - What sprint scope is locked
 - What files can be changed
 - What files must not be touched
 - What tests are required
 - What needs human approval
 - What conflicts exist
-

6. Core Philosophy

Chat is not the source of truth.

The approved project brain is the source of truth.

Chat messages can introduce ideas, changes, bugs, or clarifications, but they should not directly override approved project documents.

When a new instruction is given, AviPack should classify it as one of the following:

- New requirement
- Requirement change
- Bug fix
- Design update
- Architecture decision
- QA update
- Sprint scope change
- Release decision

Then the project brain should be updated before development continues.

7. Product Positioning

AviPack is not:

- A replacement for developers
- A replacement for project managers
- A replacement for QA engineers
- A coding model
- A framework like React, Django, Spring Boot, or Laravel

AviPack is:

- A project starter
 - A project brain generator
 - An AI development governance layer
 - A requirement and scope control system
 - A QA traceability system
 - A conflict detection system
 - An agent-agnostic instruction layer
-

8. Target Users

AviPack is useful for:

- Solo developers using AI coding agents
 - Founders building MVPs with AI assistance
 - Freelancers managing client projects
 - Software teams adopting AI development workflows
 - Technical leads supervising AI-generated code
 - QA engineers managing requirement-to-test traceability
 - Startups needing lightweight SDLC governance
 - Product teams working with fast-changing requirements
-

9. Supported AI Agents

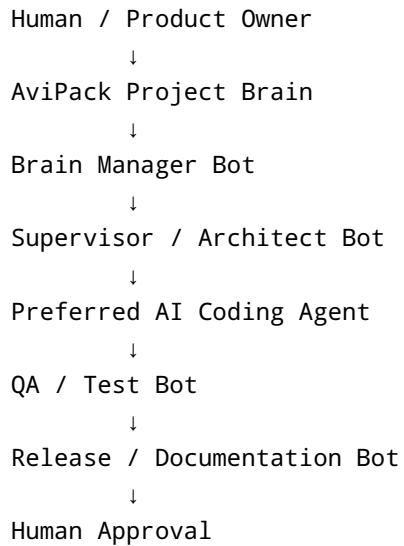
AviPack should be agent-agnostic.

It should support any preferred AI coding assistant, including:

- OpenAI Codex
- Claude Code
- Cursor
- GitHub Copilot
- Devin
- Windsurf
- Replit Agent
- Local coding agents
- Future AI development tools

AviPack should generate agent-specific instruction adapters while maintaining one shared project brain.

10. Core Architecture



11. Bot Roles

11.1 Brain Manager Bot

Responsible for maintaining the project brain.

Main responsibilities:

- Create requirement IDs
- Manage requirement versions
- Create change requests
- Run impact analysis
- Detect conflicts
- Maintain sprint scope
- Maintain traceability
- Update project brain files
- Block development when scope is unclear

11.2 Supervisor / Architect Bot

Responsible for controlling development direction.

Main responsibilities:

- Read the active sprint lock
 - Convert requirements into scoped development tasks
 - Define allowed and restricted files
 - Prevent development drift
 - Check architecture alignment
 - Decide whether human approval is needed
 - Send scoped tasks to the development agent
-

11.3 Developer Agent

Responsible for implementation.

This can be Codex, Claude Code, Cursor, GitHub Copilot, Devin, Windsurf, Replit Agent, or any other preferred AI coding tool.

Main responsibilities:

- Implement only approved requirements
 - Follow `AGENTS.md` and agent-specific instructions
 - Avoid unrelated changes
 - Run build, lint, and test commands
 - Update documentation where required
 - Produce a clear implementation summary
-

11.4 QA / Test Bot

Responsible for validating the work.

Main responsibilities:

- Check acceptance criteria
 - Map requirements to test cases
 - Run smoke tests
 - Run regression tests
 - Check UI behavior
 - Check API behavior
 - Check database behavior
 - Check security-sensitive flows
 - Report bugs
 - Approve or block release readiness
-

11.5 Release / Documentation Bot

Responsible for final release preparation.

Main responsibilities:

- Update changelog
 - Prepare release notes
 - Check Definition of Done
 - Confirm QA approval
 - Confirm no unresolved conflicts exist
 - Prepare final human approval summary
-

12. Installation Concept

The user installs AviPack into any project.

Example command:

```
npx avipack init
```

Alternative command:

```
avipack init
```

The setup wizard asks project-specific questions and generates the required project brain files.

13. Setup Wizard Questions

During setup, AviPack should ask:

```
Project name?  
Project type?  
Primary language?  
Framework?  
Package manager?  
Build command?  
Dev command?  
Lint command?  
Test command?  
E2E test command?
```

```
Git provider?  
Git branching strategy?  
Sprint length?  
Preferred AI coding agent?  
Use multiple AI agents?  
Strictness level?  
Use GitHub Actions?  
Require QA mapping?  
Require sprint locking?  
Require human approval gates?
```

Example setup:

```
Project name: Solar Monitoring Dashboard  
Project type: Full-stack web app  
Primary language: TypeScript  
Framework: Next.js  
Package manager: npm  
Build command: npm run build  
Dev command: npm run dev  
Lint command: npm run lint  
Test command: npm test  
E2E command: npm run test:e2e  
Preferred AI agent: Codex + Cursor  
Strictness level: Strict  
Sprint length: 2 weeks  
Git strategy: main/develop/feature branches
```

14. Generated Project Structure

After initialization, AviPack should generate:

```
project-root/  
  AGENTS.md  
  
  brain/  
    BRAIN_MANIFEST.yaml  
    PROJECT_CONTEXT.md  
    PRODUCT_VISION.md  
    GLOSSARY.md  
  
  requirements/  
    _template.requirement.yaml
```

```
change-requests/  
  _template.change-request.yaml  
  
designs/  
  _template.design.yaml  
  
adr/  
  _template.adr.md  
  
sprints/  
  _template.sprint.yaml  
  _template.sprint-lock.json  
  
qa/  
  TEST_STRATEGY.md  
  TEST_CASE_TEMPLATE.yaml  
  REGRESSION_CHECKLIST.md  
  RELEASE_CHECKLIST.md  
  
traceability/  
  TRACEABILITY_MATRIX.yaml  
  REQUIREMENT_CODE_MAP.yaml  
  REQUIREMENT_TEST_MAP.yaml  
  
releases/  
  _template.release.yaml  
  
backlog/  
  BACKLOG.md  
  
decisions/  
  DECISIONS.md  
  
generated/  
  brain-index.json  
  conflict-report.json  
  
agent-adapters/  
  codex/  
  claude-code/  
  cursor/  
  github-copilot/  
  devin/  
  windsurf/  
  replit/  
  
scripts/
```

```
brain-doctor.js

.github/
  workflows/
    brain-check.yml
```

15. Key Files

15.1 AGENTS.md

The generic instruction file for AI agents.

It defines:

- How AI agents should behave
 - What files to read first
 - How to identify active requirements
 - How to follow sprint locks
 - What changes need approval
 - What checks must be run
 - What output format is required
-

15.2 BRAIN_MANIFEST.yaml

The main configuration file for the project brain.

It defines:

- Project details
 - Language and framework
 - Commands
 - Active sprint
 - Source-of-truth order
 - Approval rules
 - Strictness level
 - Agent configuration
-

15.3 Requirement Files

Each requirement is stored as a versioned YAML file.

Requirement files define:

- Requirement ID
 - Title
 - Version
 - Status
 - Sprint
 - Scope
 - Out-of-scope areas
 - Acceptance criteria
 - Related design files
 - Related code areas
 - Related tests
 - Approval level
-

15.4 Change Request Files

Change requests are created when requirements change.

They define:

- Target requirement
 - Current version
 - Proposed version
 - Requested change
 - Impact analysis
 - Approval requirement
 - Decision status
-

15.5 Sprint Lock Files

Sprint locks freeze the active sprint scope.

They define:

- Sprint goal
 - Locked requirements
 - Locked requirement versions
 - Locked designs
 - Not-allowed changes
 - Definition of Done
 - Human approval conditions
-

15.6 QA Mapping Files

QA files connect requirements to tests.

They define:

- Test strategy
 - Test case templates
 - Requirement-to-test mapping
 - Regression checklist
 - Release checklist
-

16. Source of Truth Order

AviPack should enforce the following source-of-truth order:

1. Product Vision
2. Architecture Decision Records
3. Approved Requirements
4. Approved Designs
5. Active Sprint Lock
6. QA Mapping
7. Backlog
8. Chat Context

If chat instructions conflict with approved project brain files, the system should raise a conflict instead of guessing.

17. Requirement State Model

Requirements should move through clear states:

```
DRAFT
↓
REVIEWED
↓
APPROVED
↓
IN_SPRINT
↓
IN_DEVELOPMENT
↓
```

```
IN_QA
↓
DONE
↓
RELEASED
```

For requirement changes:

```
CHANGE_REQUESTED
↓
IMPACT_ANALYSIS
↓
APPROVED_CHANGE
↓
UPDATED_VERSION
```

18. Conflict Detection

AviPack should include a conflict checker called:

```
avipack doctor
```

This command should detect:

- Requirement changed without version bump
- Sprint lock points to old requirement version
- Approved requirement has no acceptance criteria
- Acceptance criteria have no mapped tests
- Change request affects DB/API/security without approval
- AI agent touched files outside approved scope
- New dependency added without approval
- Requirement conflicts with design
- Requirement conflicts with architecture decision
- Active sprint has no sprint goal
- Release includes unfinished requirements

Example output:

```
AviPack Doctor Report

Status: BLOCKED
```

Conflicts:

1. REQ-0002 is locked at v1.1.0, but current file is v1.2.0.
2. AC-004 has no mapped QA test.
3. PR modified src/auth/middleware.ts, but auth is out of scope for SPRINT-002.
4. CR-0004 affects database query behavior and requires human approval.

Recommended Actions:

- Move REQ-0002 v1.2.0 into CR-0004 or update sprint lock.
- Add QA mapping for AC-004.
- Revert auth middleware change or request approval.
- Get human approval before continuing CR-0004.

19. CLI Commands

Planned AviPack CLI commands:

```
avipack init
```

Initialize AviPack in a project.

```
avipack doctor
```

Check project brain health and conflicts.

```
avipack req new
```

Create a new requirement.

```
avipack cr new
```

Create a change request.

```
avipack sprint start
```

Create a new sprint.

```
avipack sprint lock
```

Lock requirement and design versions for a sprint.

```
avipack task create REQ-0001
```

Generate an AI-agent-ready development task.

```
avipack qa check
```

Check QA mapping and test coverage.

```
avipack release prepare
```

Prepare release notes, checklist, and release readiness report.

20. Strictness Modes

AviPack should support three modes.

Light Mode

For solo developers and early experiments.

Includes:

- Basic brain files
 - Basic AGENTS.md
 - Simple requirement templates
 - Optional QA checklist
-

Standard Mode

For normal product development.

Includes:

- Requirement versioning
- Change requests
- Sprint files
- QA mapping
- Brain doctor checks

- Agent adapter files
-

Strict Mode

For serious production projects.

Includes:

- Sprint locking
 - Approval gates
 - Requirement-to-test traceability
 - ADR enforcement
 - Conflict blocking
 - Release readiness checks
 - GitHub Action validation
-

21. Agent Adapter Layer

AviPack should generate instruction files for the selected AI agents.

Examples:

```
AGENTS.md
CLAUDE.md
.cursor/rules/project-brain.mdc
.github/copilot-instructions.md
.devin/instructions.md
.windsurf/rules/project-brain.md
.replit/agent-instructions.md
```

The core brain remains the same.

Only the adapter changes based on the selected agent.

22. MVP Scope

The first MVP should include:

1. `avipack init`
2. Guided setup wizard
3. `AGENTS.md` generator

4. `BRAIN_MANIFEST.yaml` generator
 5. Requirement template generator
 6. Change request template generator
 7. Sprint template generator
 8. Sprint lock template generator
 9. QA checklist generator
 10. Traceability template generator
 11. Basic `avipack doctor`
 12. Basic AI task generator
-

23. Future Features

Future versions can include:

- Web dashboard
 - VS Code extension
 - GitHub App
 - GitHub Issues sync
 - Jira integration
 - Linear integration
 - Notion integration
 - Requirement diff viewer
 - Sprint scope visualizer
 - QA coverage dashboard
 - Release readiness dashboard
 - Agent performance tracking
 - Multi-agent orchestration
 - Team approval workflow
-

24. Security Principles

AviPack should be safe by design.

Initial rules:

- Do not ask for AI API keys
- Do not ask for GitHub tokens
- Do not upload project files anywhere
- Run locally by default
- Write only generated brain files
- Show file changes before applying where possible
- Keep dependencies minimal
- Do not read `.env` files unless explicitly allowed
- Do not expose secrets in generated reports

25. Product Differentiation

AviPack is different from coding agents.

Coding agents answer:

How do we write the code?

AviPack answers:

What should the agent build, under which requirement version, within which sprint scope, with which QA rules, and under which approval conditions?

AviPack is the missing control layer between human intention and AI-generated implementation.

26. Final Product Vision

AviPack helps move software development from:

Human constantly guides AI agent

to:

Human defines product direction
AviPack maintains the project brain
AI agent develops inside approved scope
QA bot validates the result
Human approves only important decisions

Final vision:

AviPack becomes the universal project brain pack for controlled, agent-agnostic, AI-assisted software development.